

Lecture Note: Phase 3 – Iteration & Automation

1. Introduction to Loops

Loops allow you to execute a block of code multiple times. This is essential for handling large datasets or repeating tasks until a specific goal is reached.

2. The while Loop

Repeats as long as a condition remains **True**.

- **Key Components:** You must have a way to change the condition (e.g., an increment) to avoid an **infinite loop**.

Syntax:

```
Python
count = 0
while count < 5:
    print(count)
    count += 1 # Increment to eventually break the loop
```

3. The for Loop

Used for iterating over a sequence (like a string or a list) or a range of numbers. It is generally more "Pythonic" and safer than a while loop for fixed iterations.

- **The range () Function:** Generates a sequence of numbers.
 - `range(5)` $\rightarrow$ 0, 1, 2, 3, 4
 - `range(1, 6)` $\rightarrow$ 1, 2, 3, 4, 5
 - `range(0, 10, 2)` $\rightarrow$ 0, 2, 4, 6, 8 (Starts at 0, steps by 2)

Example:

```
Python
for i in range(3):
    print("Repeating...")
```

4. Loop Control Statements

Sometimes you need to change the behavior of a loop from the inside:

- **break:** Immediately terminates the loop and moves to the next section of code.
- **continue:** Skips the rest of the current iteration and jumps back to the start of the next one.

5. Nested Loops

A loop inside another loop. The "inner" loop completes all its iterations for every single iteration of the "outer" loop.

- **Common Use:** Generating grids, coordinates, or processing multi-dimensional data.

```
Python
for i in range(2):          # Outer loop
    for j in range(2):      # Inner loop
        print(f"({i}, {j})")
```

Summary Table

Feature	While Loop	For Loop
Usage	When the number of iterations is unknown.	When iterating over a known sequence or range.
Control	Depends on a condition.	Depends on the length of the sequence.
Risk	High risk of infinite loops if logic is flawed.	Generally safer and cleaner for sequences.